Chapter 1
Introduction

Prof. Dr. Khalid M. Hosny

Professor of Information Technology
Vice Dean for Community Services
Faculty of Computers & Informatics
Zagazig University

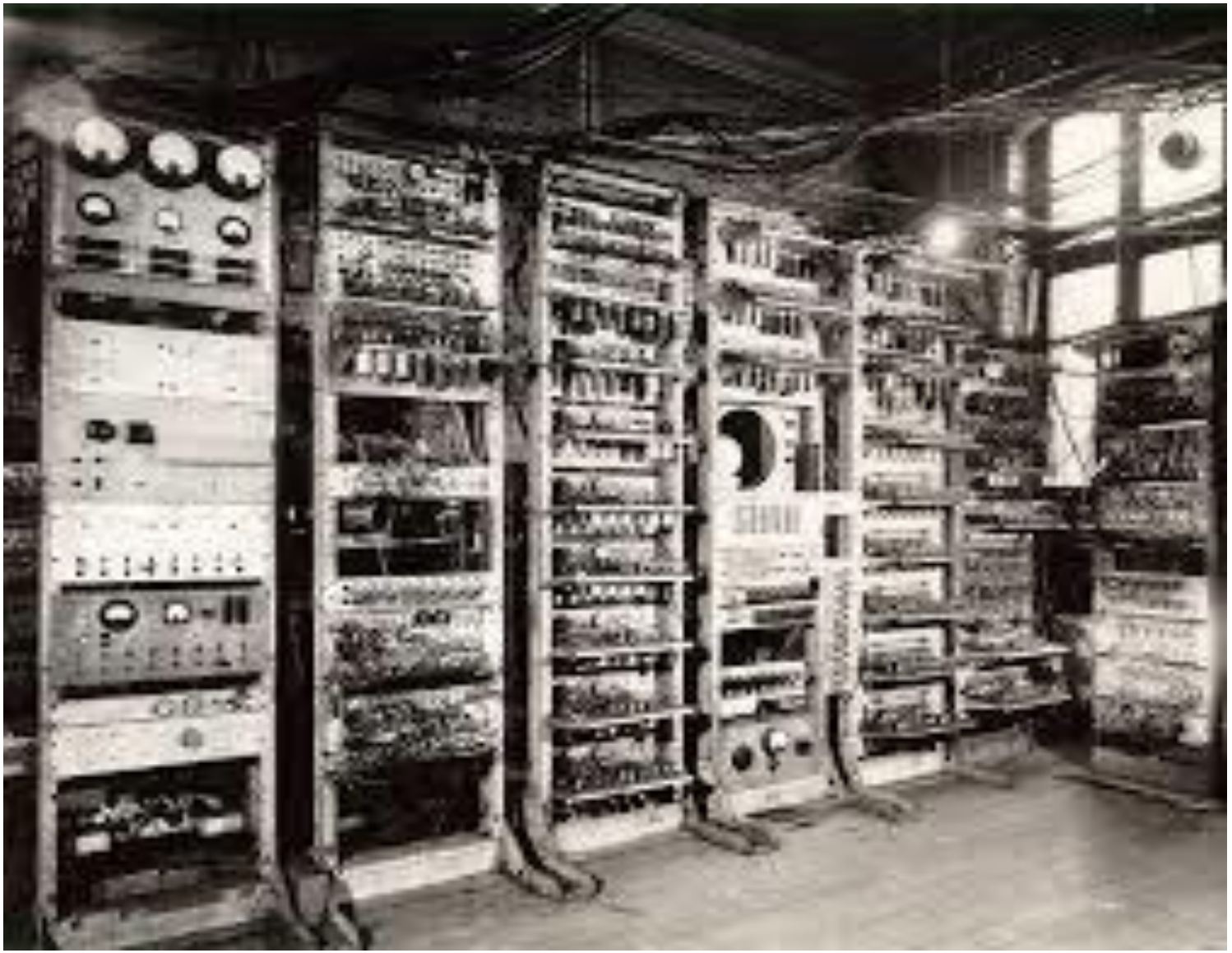
Computer Generations

- Computers are primarily electronic devices whose size and ability to perform various functions were greatly affected by the state of electronics at the time of manufacturing.
- More precisely, if electronics in a certain period of time are large, slow and unreliable, it is normal for computers in this period to be large, slow and unreliable.
- Accordingly, computers were divided into four generations depending on the development of electronics in that time period.

First Computer-Generation:

- Computers were the first-generation industry in the period between **1942** and **1955**.
- In this period, the manufactures of the computer hardware used the **vacuum tubes** and **electro-mechanical devices**.
- Despite the large size of the vacuum tubes their performance is very bad.

- The computers' sizes were very large.
- Very slow.
- Operation of these computers results in a lot of heat.
- These devices need permanent cooling.
- A limited number of scientists are able to operate these devices.
- Requires regular maintenance which increases operating cost.



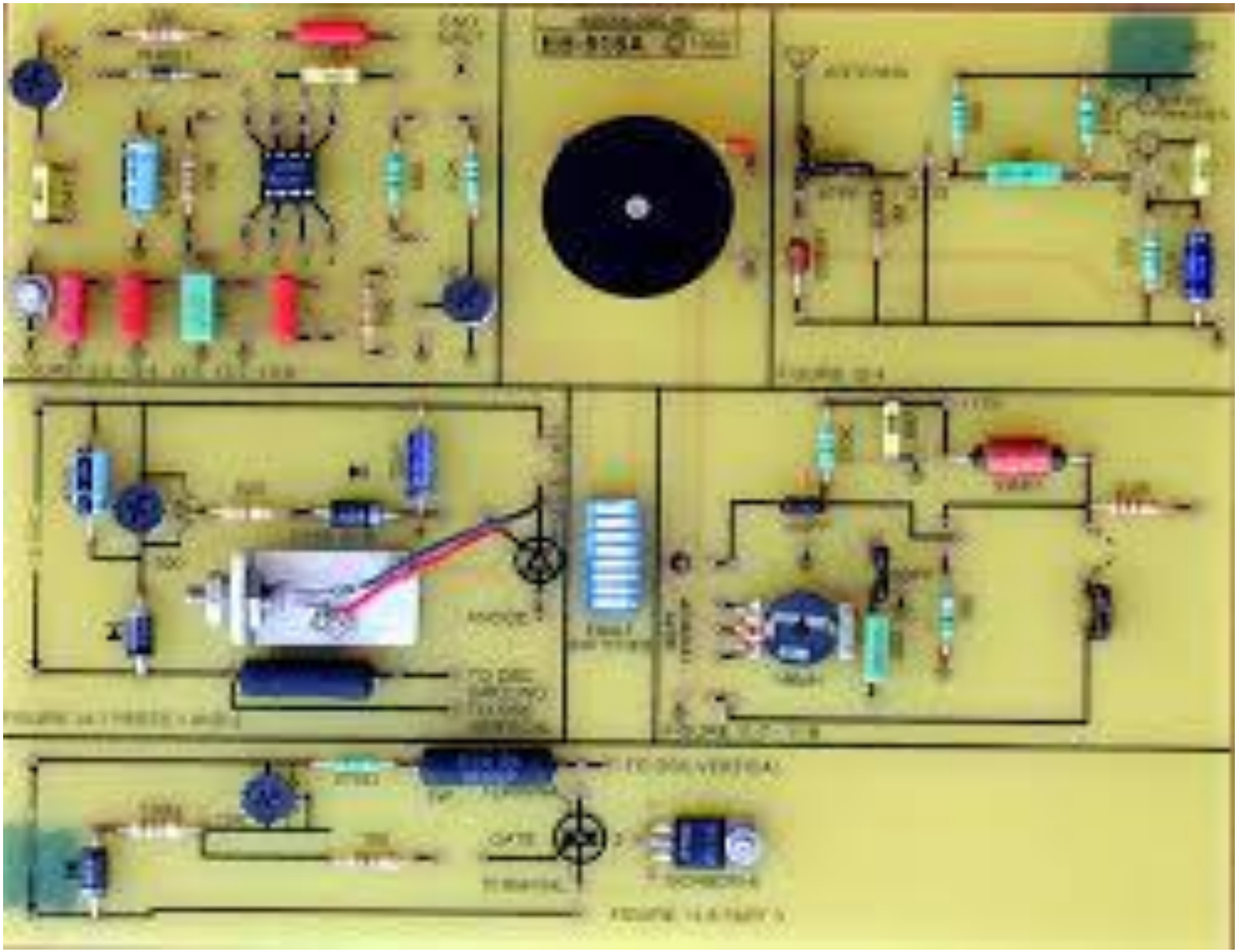
A courtesy image of the Mark-1

Second Computer-Generation:

- The second-generation of computer industry began in the middle of **1950s**.
- The transistor (also is called Semiconductor) was much smaller than the vacuum tubes.
- Its operation reduced the emitted heat.
- Therefore, this generation's devices overcome on many problems associated with the first generation of computers.
- The computers of this generation were able to run some high-level programming languages such as **Fortran**.

- Despite their small size and efficiency, the second-generation computers suffered from some technical problems which could be summarized as follows:

  - ✓ Computer sizes were relatively large.
  - ✓ Execution speeds were slow.
  - ✓ Limited people were able to use.



A courtesy image of the transistor

Third Computer-Generation:

- Production of Integrated Circuits (IC) represents a new turning point in the computer industry.
- The manufacturing of third-generation of computers began in the second half of 1960s and continuing for one decade.
- The main advantage of integrated circuit technology is that, a number of circuits could be integrated into a very small of flat area of silicon, which is called a **chip**.
- This technology led to the fact that the third-generation computers are about 1000 times faster than the computers of the first-generation.

- Logical operations needs a nanosecond only where one second is equivalent to 10^9 nano second.
- With the third-generation of computers we are no longer talking about defects, but began to talk about the advantages.
- The most important of these advantages are:
  - ✓ High speed,
  - ✓ Small size
  - ✓ The ability to execute a number of programs at the same time.

Fourth Computer-Generation

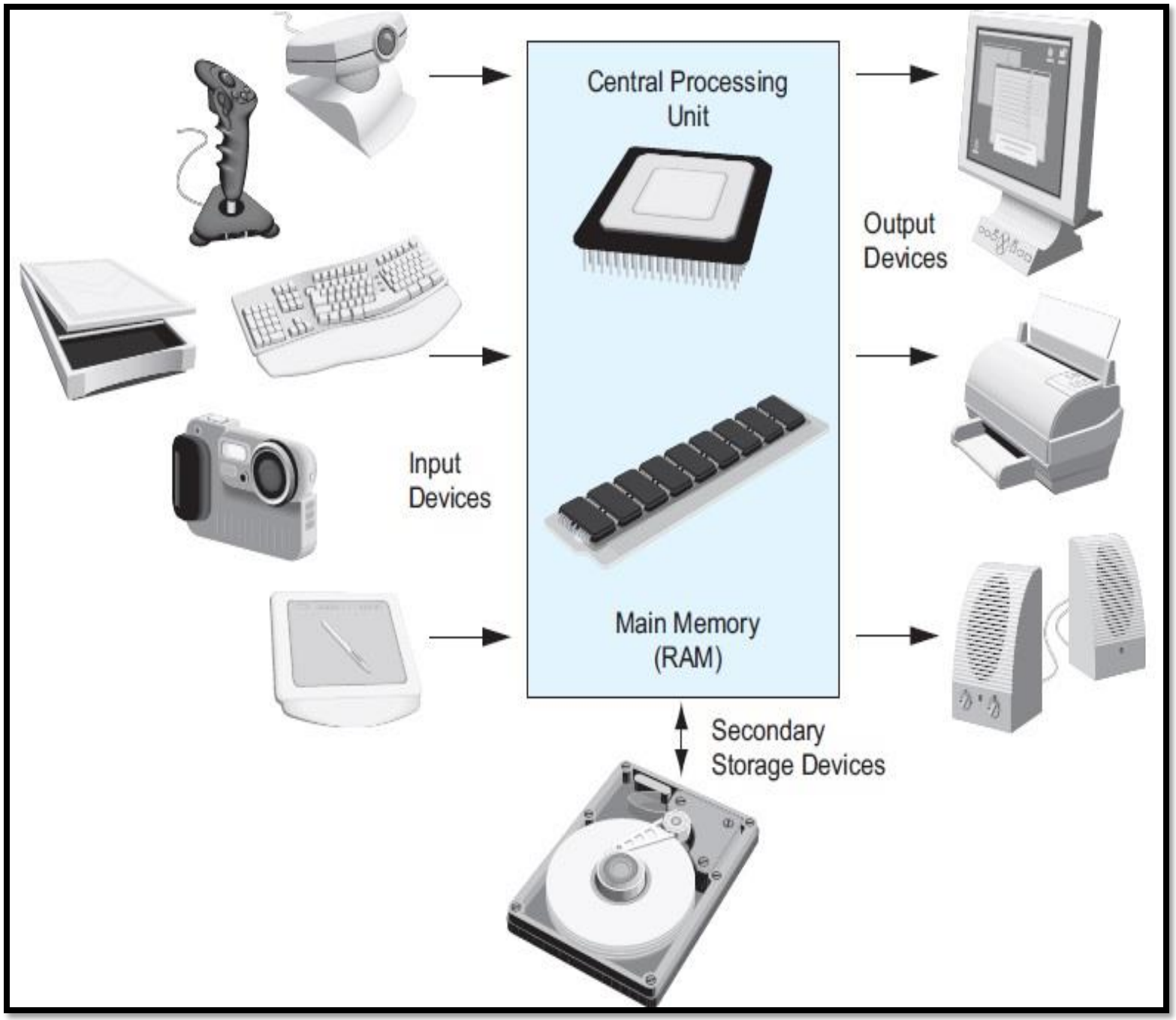
- The great development of the integrated circuit technology led to the development of so-called microprocessor chipsets which contains the entire **Central Processing Unit** (CPU) in one silicon chip.
- This development in of computer industry formed the fourth-generation.
- It started in the middle of 1970s and still the main core of computer industry till this moment.
- Over time, **thousands of circuits could be grouped on the same chip** which increased the capabilities of the produced computers.



IBM-Compatible PC



An image for PC and its Peripherals


Central Processing Unit
Output Devices
Input Devices
Main Memory (RAM)
Secondary Storage Devices

Important Terms

Data:

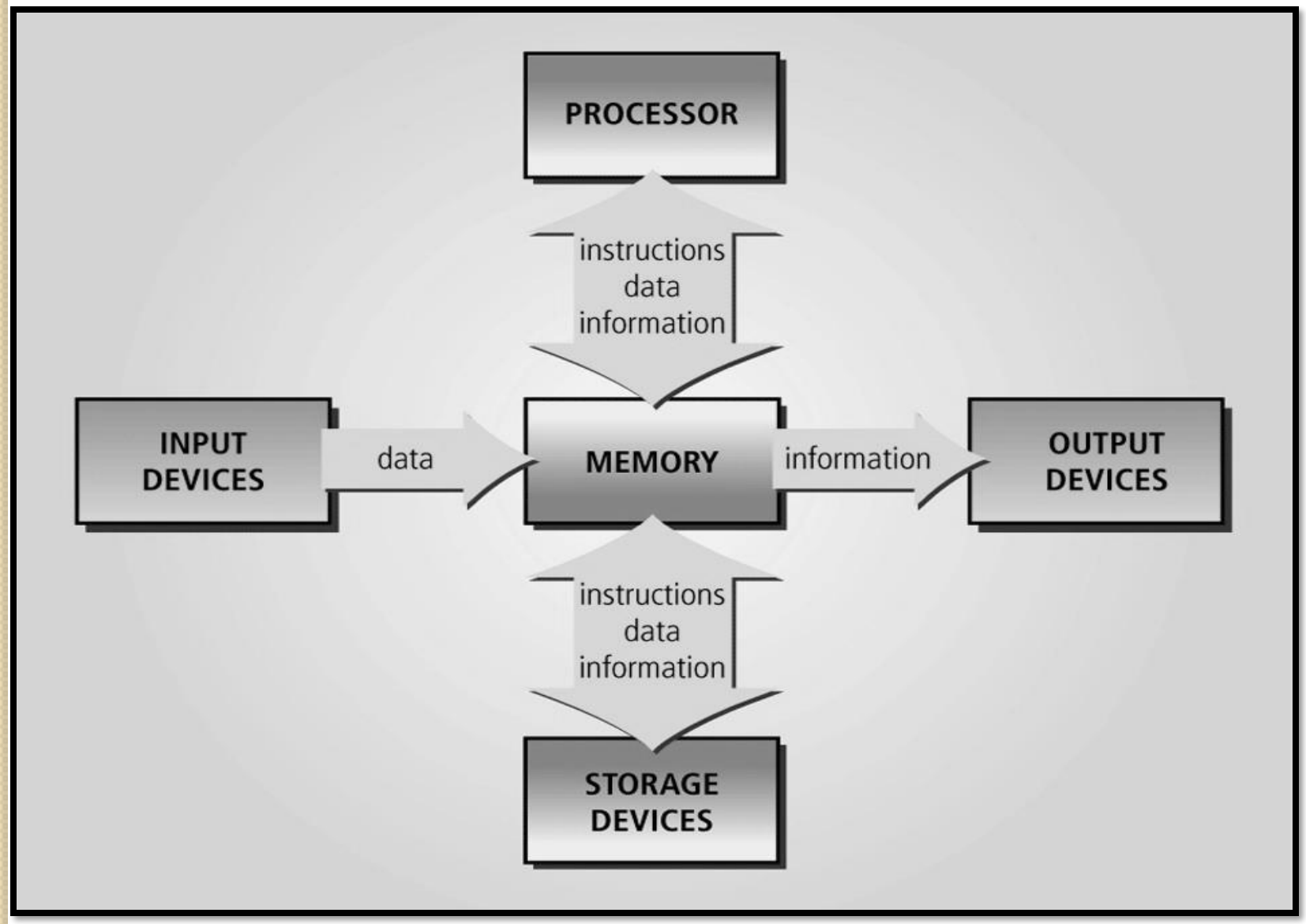
Any raw information that can be entered to the computer by any of the input methods.

Processing:

The process of converting and analyzing data into meaningful useful information

Information:

is the processed data which formulated in a meaningful form.



Data Processing steps

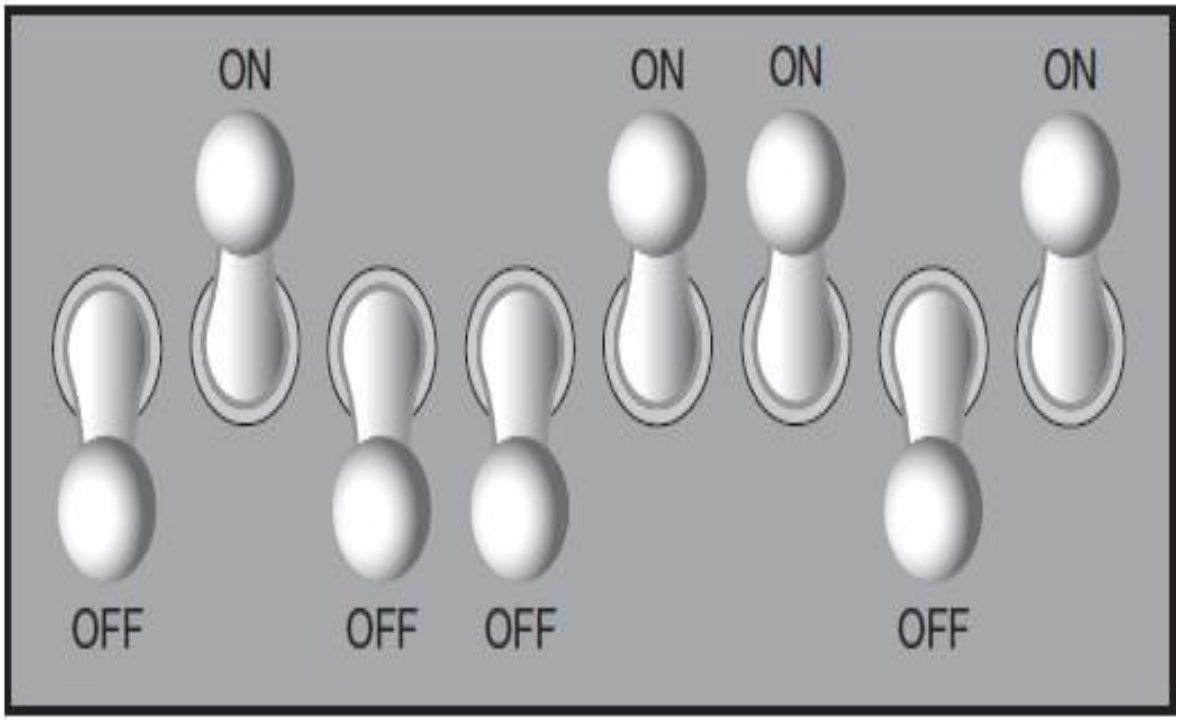
Types of Data

- Text
- Images
- Audio
- Video

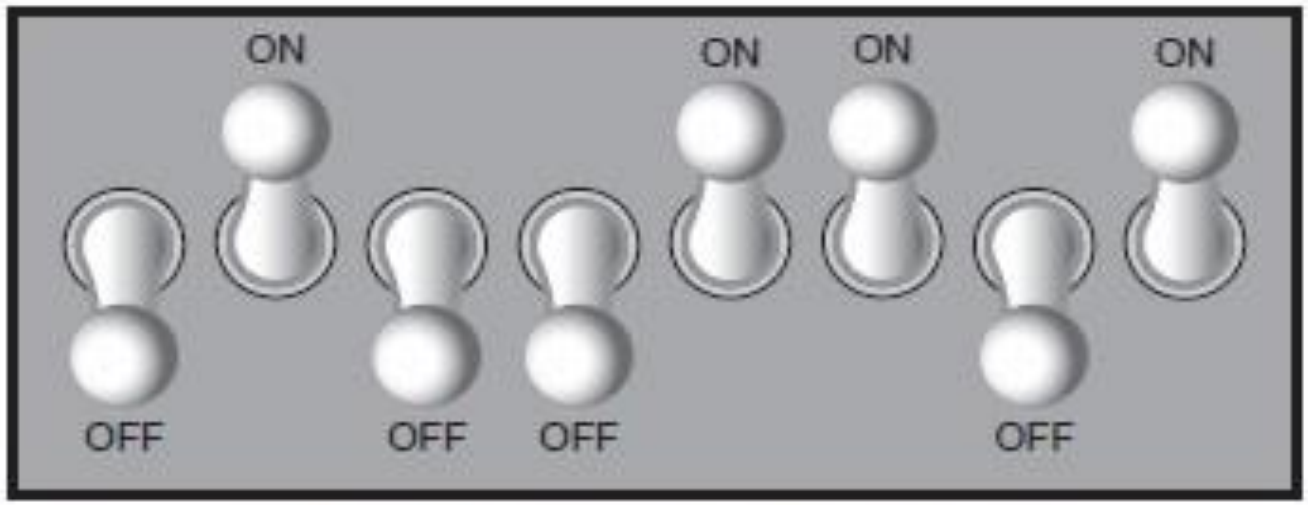
- **All data entered into the computer is converted to numbers and then converted to binary**
- **so all types of data must be converted from different forms Into binary numbers to be handled inside the computer.**

How Computers Stores Data

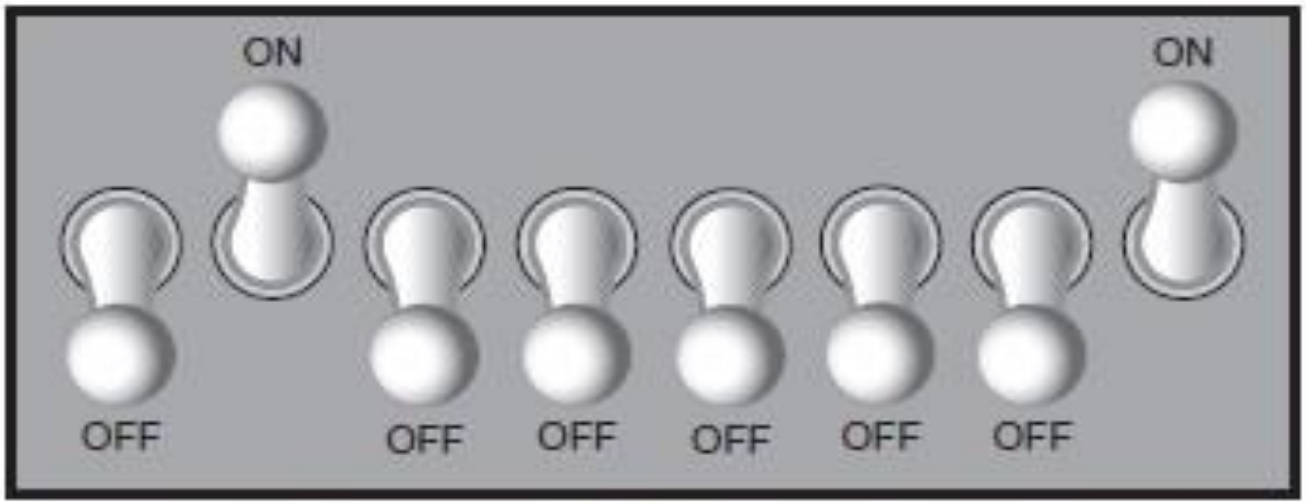
1. Computer memory is divided into storage areas called bytes
2. Each of these bytes represents enough memory to store a small number or a single character
3. to store something meaningful that we need a large number of bytes
4. Today's computers have millions or even billions of bytes available for storage



A description of one byte of memory



The number 77 stored in a byte.



The letter A stored in a byte.

- All the input numbers are converted to the computer and represented in the Binary Numbering System
- The process of representing numbers using the binary system depends on the position of each number and the value assigned to it
- The smaller value is always at the far right and the value increases as we move to the left
- The positions of the allocated numbers are in the form of the system base (Radix) raised to a positive integer's exponent

Digital Signals

- Digital Signals have two basic states:

  1 (logic "high", or H, or "on")

  0 (logic "low", or L, or "off")

- Digital values are in a *binary* format. Binary means 2 states.

- A good example of binary is a light:
- (only <u>on</u> or <u>off</u>)



Digital Systems and Binary Numbers

- ***Digital computers***
  - General purposes
  - Many scientific, industrial and commercial applications
- ***Digital systems***
  - Digital camera
  - Electronic calculators, PDA's
  - Digital TV

Binary

In Binary, there are only 0's and 1's. These numbers are called "Base-2" (Example: 010_2)

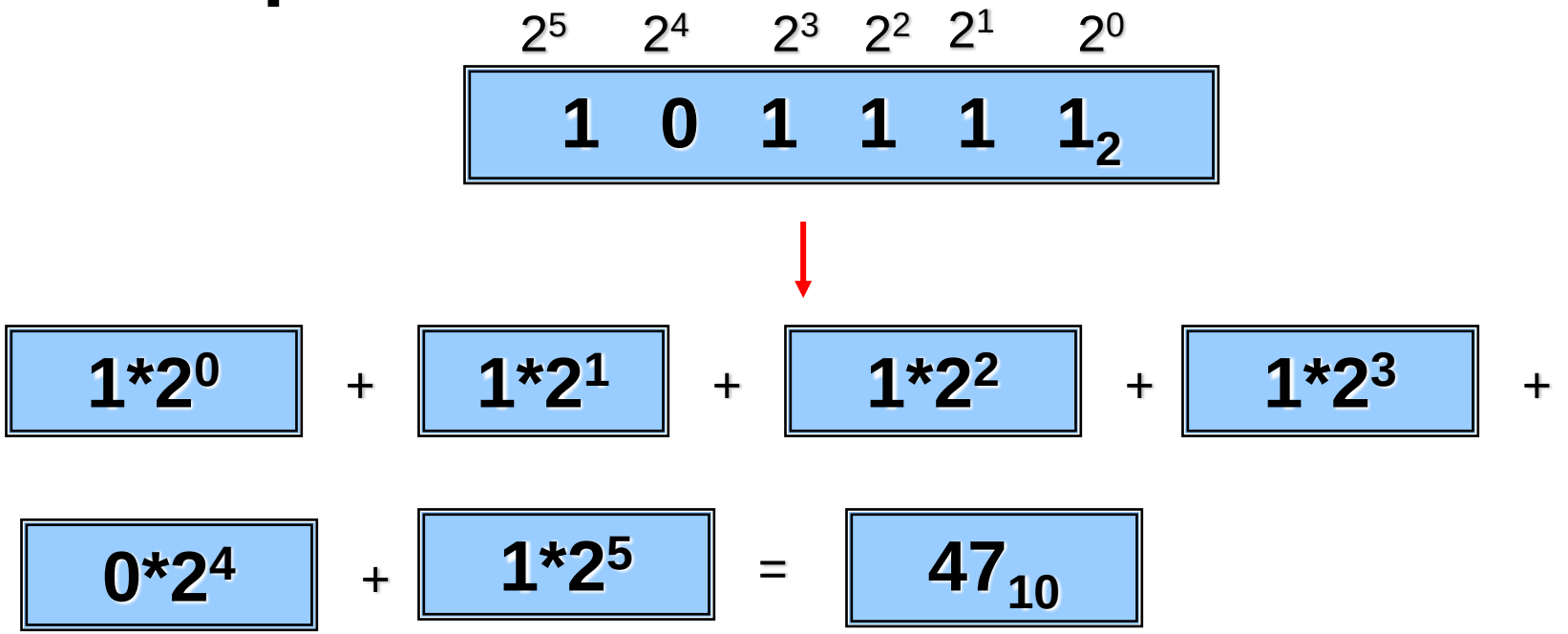
We count in "Base-10" (0 to 9)

Binary to Decimal

Base 2	=	Base 10
000	=	0
001	=	1
010	=	2
011	=	3
100	=	4
101	=	5
110	=	6
111	=	7

Number systems

- **Converting from binary to decimal :**
  - Evaluate the power series
- **Example**



Number systems

- **Convert from binary to decimal :**

  1011011

  a. 27
  b. 91
  c. 109
  d. -109
  e. 551

Review of Number systems

Memorize the first ten powers of two

Table 1.2 Powers of 2.

n	2^n	n	2^n
1	2	11	2,048
2	4	12	4,096
3	8	13	8,192
4	16	14	16,384
5	32	15	32,768
6	64	16	65,536
7	128	17	131,072
8	256	18	262,144
9	512	19	524,288
10	1,024	20	1,048,576

Review of Number systems

Table 1.3 First 32 binary integers.

Decimal	Binary	4-bit	Decimal	Binary
0	0	0000	16	10000
1	1	0001	17	10001
2	10	0010	18	10010
3	11	0011	19	10011
4	100	0100	20	10100
5	101	0101	21	10101
6	110	0110	22	10110
7	111	0111	23	10111
8	1000	1000	24	11000
9	1001	1001	25	11001
10	1010	1010	26	11010
11	1011	1011	27	11011
12	1100	1100	28	11100
13	1101	1101	29	11101
14	1110	1110	30	11110
15	1111	1111	31	11111

Note that the number one less than 2^n consists of n 1's (for example, $2^4 - 1 = 1111 = 15$ and $2^5 - 1 = 11111 = 31$).

Number systems

- **Converting from decimal to binary :**
  - Divide the decimal number by 2 repeatedly.
  - The remainder gives the digits of the binary number

2	746	
2	373 R	0
2	23 R	0
2	11 R	1
2	5 R	1
2	2 R	1
2	1 R	0
2	0 R	1

$746_{10} = 1011101010_2$

- **Convert from decimal to binary :**

65

a.110101

b.101110

c.100001

d.100000

e.1000001

Examples

- Convert the following to base 10 : 11.01100_2

  a. 3.7500
  b. 3.0300
  c. 3.1875
  d. 3.0300
  e. 3.3750

BCD

Binary coded decimal (BCD) is a weighted code that is commonly used in digital systems when it is necessary to show decimal numbers such as in clock displays.

The table illustrates the difference between straight binary and BCD.

BCD represents each decimal digit with a 4-bit code.

Notice that the codes 1010 through 1111 are not used in BCD.

Decimal	Binary	BCD
0	0000	0000
1	0001	0001
2	0010	0010
3	0011	0011
4	0100	0100
5	0101	0101
6	0110	0110
7	0111	0111
8	1000	1000
9	1001	1001
10	1010	0001 0000
11	1011	0001 0001
12	1100	0001 0010
13	1101	0001 0011
14	1110	0001 0100
15	1111	0001 0101

ASCII

- ASCII is a code for alphanumeric characters and control characters.
- In its original form, ASCII encoded 128 characters and symbols using 7-bits.
- The first 32 characters are control characters, that are based on obsolete teletype requirements, so these characters are generally assigned to other functions in modern usage.
- In 1981, IBM introduced extended ASCII, which is an 8-bit code and increased the character set to 256.
- Other extended sets (such as Unicode) have been introduced to handle characters in languages other than English.

Hexadecimals – Base 16

Hexadecimal is a weighted number system. The column weights are powers of 16, which increase from right to left.

Column weights	16^3	16^2	16^1	16^0.
	4096	256	16	1 .

Example Express $1A2F_{16}$ in decimal.

Start by writing the column weights:

4096	256	16	1
1	A	2	F_{16}

$1(4096) + 10(256) + 2(16) + 15(1) = 6703_{10}$

Decimal	Hexadecimal	Binary
0	0	0000
1	1	0001
2	2	0010
3	3	0011
4	4	0100
5	5	0101
6	6	0110
7	7	0111
8	8	1000
9	9	1001
10	A	1010
11	B	1011
12	C	1100
13	D	1101
14	E	1110
15	F	1111

Hexadecimals – Base 16

- Shorthand for binary
- Binary digits are grouped into 4
  - Start at the least significant
  - If number of digits is not a multiple of 4, add zeros
- Each group is interpreted in decimal
- Digits above 9 are represented by the first six letter of the alphabet:
  - 10: A; 11: B; 12: C; 13: D; 14: E; 15: F
- Example:

$$10111101010_2 = 0010\ 1110\ 1010_2$$
$$= 2EA_{16}$$

Hexadecimals – Base 16

- **Convert from binary to hexadecimal :**

1111111

a.771

b.177

c.F7

d.7F

e.127

Hexadecimals – Base 16

What is the binary equivalent of $(24C)_{16}$?

Solution

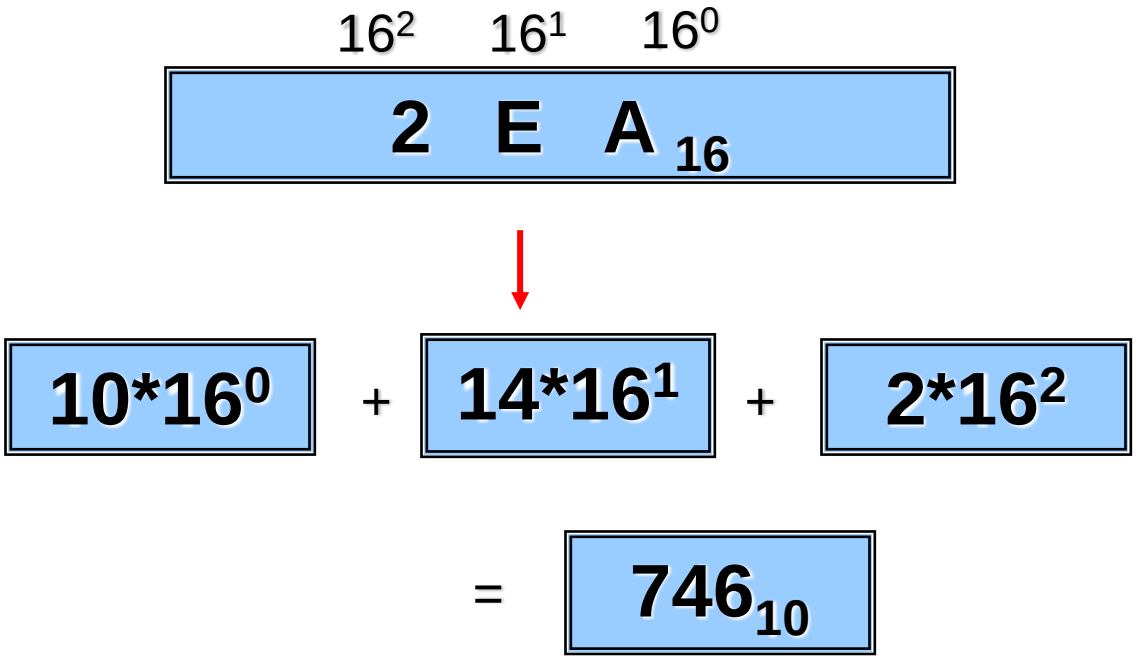
Each hexadecimal digit is converted to 4-bit patterns:

$$2 \rightarrow 0010$$
$$4 \rightarrow 0100$$
$$C \rightarrow 1100$$

The result is $(001001001100)_2$.

Hexadecimals – Base 16

- **Converting from hex to decimal :**
  - Evaluate the power series
- **Example**



Hexadecimals – Base 16

- **Convert from hexadecimal to decimal :**

65

a.65

b.101

c.86

d.100001

e.41

Converting Decimal to Hex

Example:

We want to convert 125_{10} to hex.

```
125 / 16 = 7 R 13
 7 / 16 = 0 R  7
```



$125_{10} = 7D_{16}$

Base 16
Cheat Sheet

$A_{16} = 10_{10}$
$B_{16} = 11_{10}$
$C_{16} = 12_{10}$
$D_{16} = 13_{10}$
$E_{16} = 14_{10}$
$F_{16} = 15_{10}$

Octal – Base 8

Octal uses eight characters the numbers from 0 through 7 to represent numbers.

Binary number can easily be converted to octal by grouping bits 3 at a time and writing the equivalent octal character for each group.

Decimal	Octal	Binary
0	0	000
1	1	001
2	2	010
3	3	011
4	4	100
5	5	101
6	6	110
7	7	111

Example

Express $1\ 001\ 011\ 000\ 001\ 110_2$ in octal:

Group the binary number by 3-bits starting from the right. Thus, 113016_8

Octal – Base 8

- Same steps as for conversion as binary and hexadecimal and any other base

- **Converting from decimal to octal :**
  - Divide the decimal number by 8 repeatedly.
  - The remainder gives the digits of the binary number

- Example: Convert 56_{10} to base 8.

Octal – Base 8

- **Converting from decimal to octal :**

Octal

	Quotient	Remainder
$56 \div 8 =$	7	0
$7 \div 8 =$	0	7

$56_{10} = 111000_2$

$56_{10} = 70_8$

Octal – Base 8

- **Convert from decimal to octal :**

15

a.71

b.177

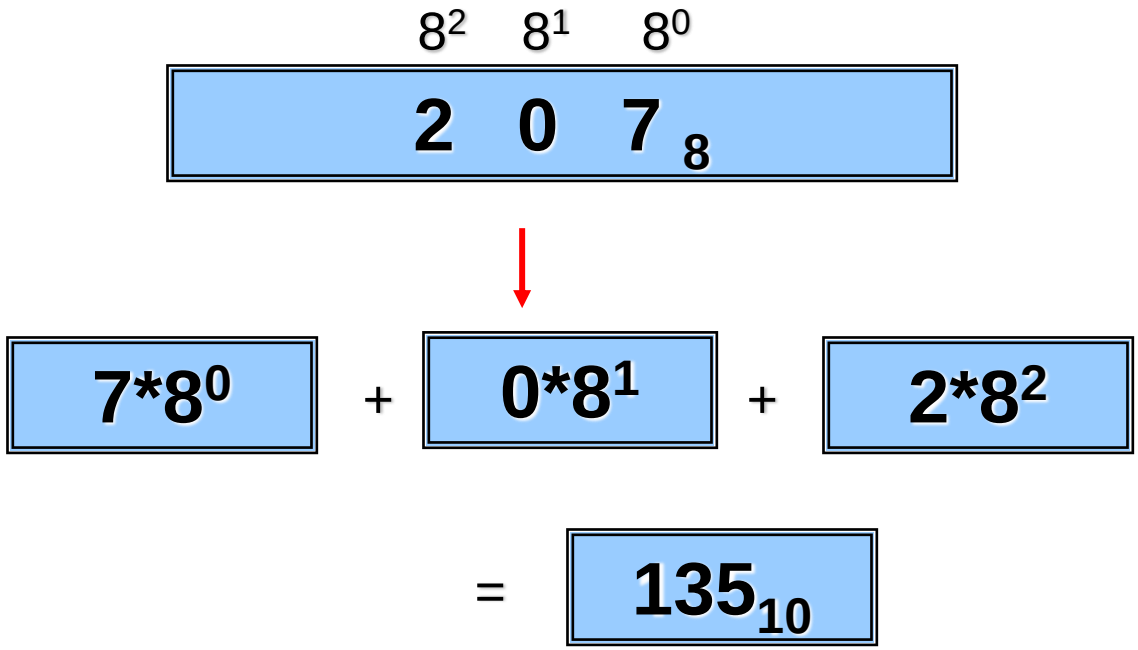
c.F7

d.17

e.27

Octal

- **Converting from octal to hexadecimal:**
  - Evaluate the power series, convert the number to decimal first.
- **Example**



Octal

Converting Binary to Octal

start

$$10001100101001_2$$

- Take the binary number and from right to left.
- Group all placeholders in triplets.
- Add leading zeros, if necessary.

010	001	100	101	001
2	1	4	5	1

$$010\ 001\ 100\ 101\ 001_2 = 21451_8$$

Converting Octal to Binary

43520_8

- Take each octal digit and convert each digit to a binary triplet.
- Keep leading zeros

4	3	5	2	0
100	011	101	010	000

$43520_8 = 100011101010000_2$

Summary

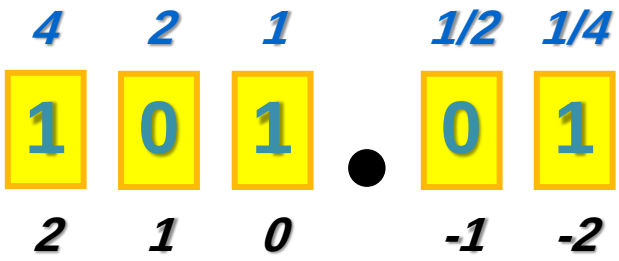
- Bases
  - Binary: Base 2
  - Octal: Base 8
  - Decimal: Base 10
  - Hexadecimal: Base 16
- Positional number system
  - $101_2 = 1\times2^2 + 0\times2^1 + 1\times2^0$
  - $63_8 = 6\times8^1 + 3\times8^0$
  - $A1_{16} = 10\times16^1 + 1\times16^0$

Summary

- Conversion from binary to octal/hex
  - Binary: 10011110001
  - Octal: 10 | 011 | 110 | 001=2361_8
  - Hex: 100 | 1111 | 0001=$4F1_{16}$
- Conversion from binary to decimal
  - $101_2 = 1\times2^2 + 0\times2^1 + 1\times2^0 = 5_{10}$
  - $63.4_8 = 6\times8^1 + 3\times8^0 + 4\times8^{-1} = 51.5_{10}$
  - $A1_{16} = 10\times16^1 + 1\times16^0 = 161_{10}$

Binary: Fraction

- Base = 2
  - 2 digits { 0, 1 }, called *b*inary dig*its* or "*bits*"
- Weights
  - Weight = (*Base*) ^{Position}
- Magnitude
  - Sum of "*Bit* x *Weight*"



$$1*2^{0}+0*2^{1}+1*2^{2}+0*2^{-1}+1*2^{-2}$$

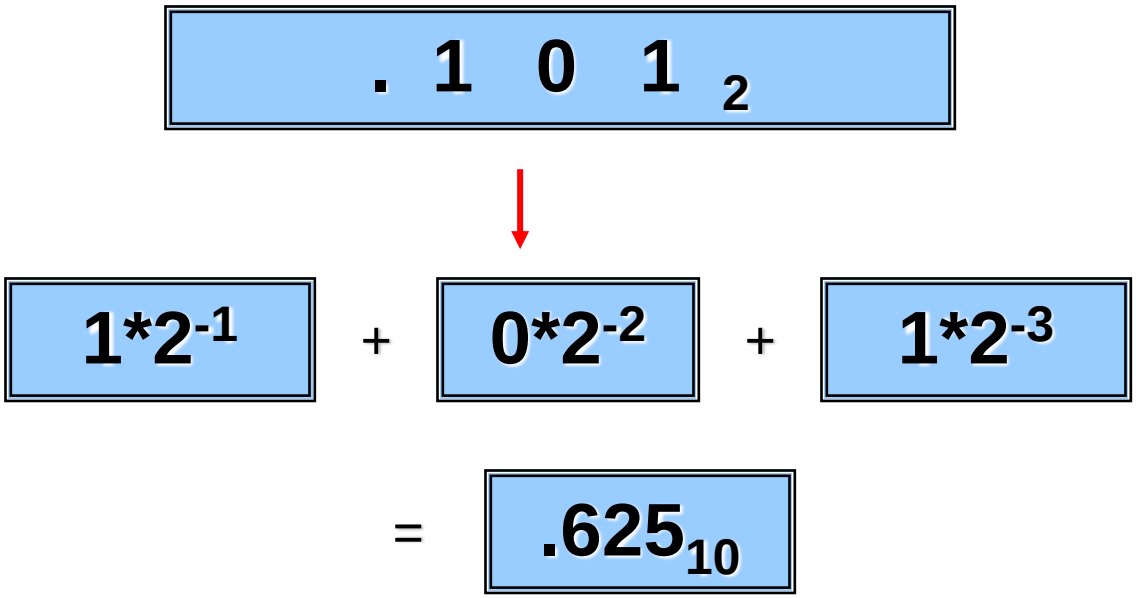
$$(101.01)_2 = (5.25)_{10}$$

Fractions

- **Converting fractions from binary to decimal :**

$$a_{-1}r^{-1} + a_{-1}r^{-2} + a_{-1}r^{-3} \ldots$$

- **Example**



Fractions

- Convert $.0111_2$ to decimal

a. .875
b. .375
c. .4375
d. .0700
e. 4.375

Fractions

- **Converting from decimal to binary :**
  - Multiply the decimal number by 2 repeatedly.
  - Use the integer part as the next digit each time, and then discard the integer
  - When the fraction part is zero, we have an exact conversion
  - Add trailing zeros to obtain the desired size

.625*2 = 1.25	**.1**
.25*2 = 0.50	**.10**
.5*2 = 1.00	**.101**

  - For some fractions, we never get an exact conversion because the fraction parts repeats, example: .3

Examples

Convert the following to base 2 : $.75_{10}$

a. .111000
b. .000011
c. .110000
d. .111111
e. .101000

Hexadecimals – Base 16: fraction

- Form Hexa to decimal
- Base = 16
  - 16 digits { 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, A, B, C, D, E, F }
- Weights
  - Weight = (*Base*) ^{Position}
- Magnitude
  - Sum of "*Digit* x *Weight*"

256	16	1		1/16	1/256
1	E	5	.	7	A
2	1	0		-1	-2

$$1*16^{2}+14*16^{1}+5*16^{0}+7*16^{-1}+10*16^{-2}$$

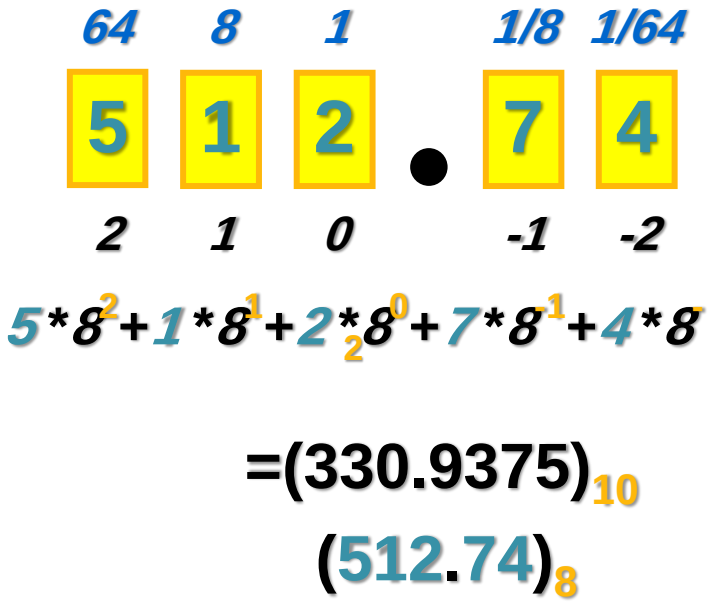
$$=(485.4765625)_{10}$$

$$(1E5.7A)_{16}$$

Octal: fraction

From octal to decimal

- Base = 8
  - 8 digits { 0, 1, 2, 3, 4, 5, 6, 7 }
- Weights
  - Weight = (*Base*) ^{Position}
- Magnitude
  - Sum of "*Digit* x *Weight*"
- Formal Notation



$=(330.9375)_{10}$

$(512.74)_8$

Binary Addition

The rules for binary addition are

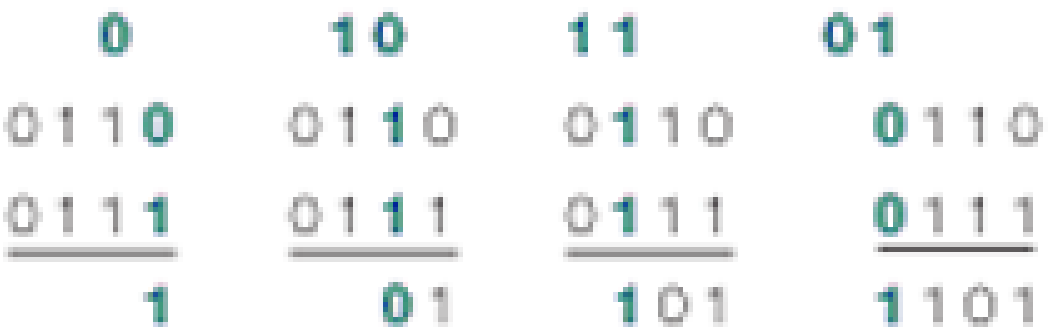
Sum = 0, carry = 0	0 + 0
Sum = 1, carry = 0	0 + 1
Sum = 1, carry = 0	1 + 0
Sum = 0, carry = 1	1 + 1

When an input carry = 1 due to a previous result, the rules are

Sum = 1, carry = 0	1 + 0 + 0 = 01
Sum = 0, carry = 1	1 + 0 + 1 = 10
Sum = 0, carry = 1	1 + 1 + 0 = 10
Sum = 1, carry = 1	1 + 1 + 1 = 11

Binary Addition

- Add one digit at a time
- Obtain a sum and a carry
- Similar to decimal addition – but pay attention to the base



Binary Addition

- Add the following binary number
- 10011+11111

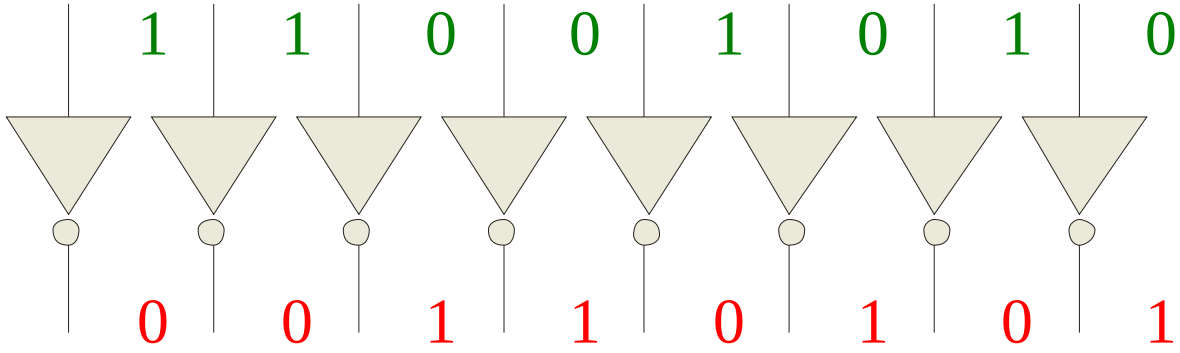
a. 110010
b. 001100
c. 101110
d. 021120
e. 010011

1’s Complement

The 1’s complement of a binary number is just the inverse of the digits. To form the 1’s complement, change all 0’s to 1’s and all 1’s to 0’s.

For example, the 1’s complement of 11001010 is 00110101

In digital circuits, the 1’s complement is formed by using inverters:



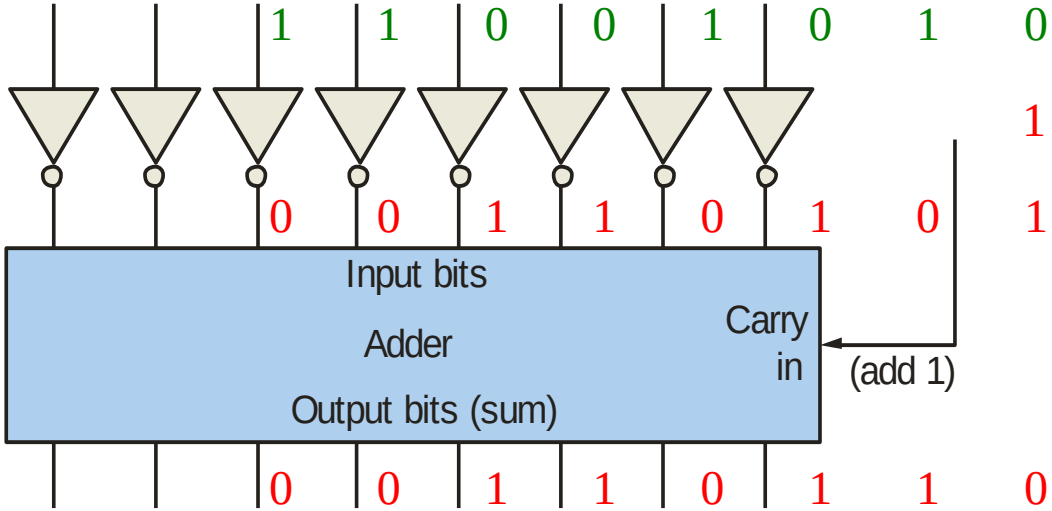
2's Complement

The 2's complement of a binary number is found by adding 1 to the LSB of the 1's complement.

Recall that the 1's complement of 11001010 is 00110101 (1's complement)

To form the 2's complement, add 1: +1

00110110 (2's complement)



Signed Numbers

Signed Binary Numbers

There are several ways to represent signed binary numbers. In all cases, the MSB in a signed number is the sign bit, that tells you if the number is positive or negative.

Computers use a modified 2's complement for signed numbers. Positive numbers are stored in *true* form (with a 0 for the sign bit) and negative numbers are stored in *complement* form (with a 1 for the sign bit).

For example, the positive number 58 is written using 8-bits as 00111010 (true form).

Sign bit

Magnitude bits

Signed Numbers

Signed Binary Numbers

Negative numbers are written as the 2's complement of the corresponding positive number.

The negative number –58 is written as:

–58 = 11000110 (complement form)

Sign bit — Magnitude bits

An easy way to read a signed number that uses this notation is to assign the sign bit a column weight of –128 (for an 8-bit number). Then add the column weights for the 1's.

Example Assuming that the sign bit = –128, show that 11000110 = –58 as a 2's complement signed number:

Column weights: –128 64 32 16 8 4 2 1.

1 1 0 0 0 1 1 0

–128 +64 +4 +2 = –58

Signed Numbers

- Derive the 5-bit binary negative (two's complement) of 17

a. 0101111
b. 101111
c. 10000
d. 01111
e. 01110

Overflow

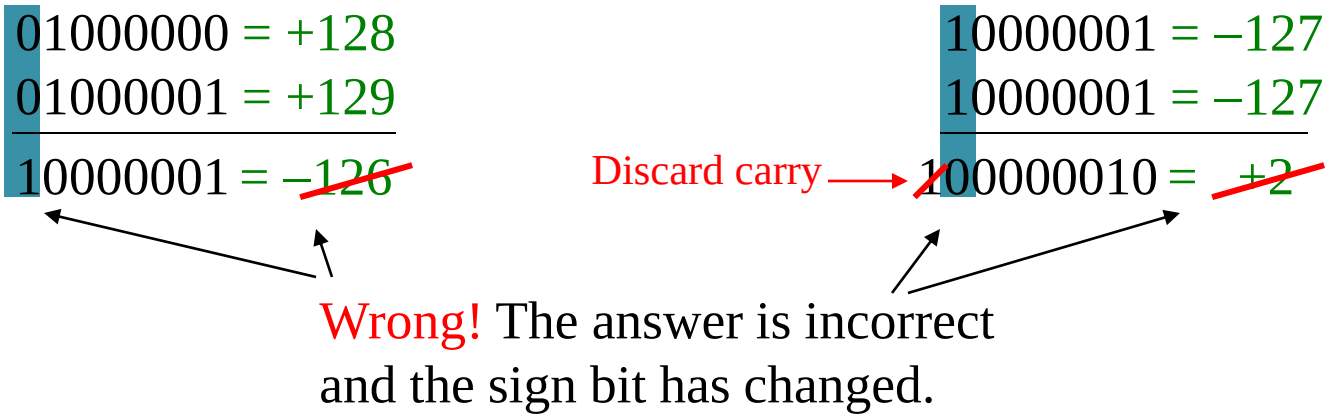
- This occurs when the sum is out of range

- Example: for 4-bit numbers, the range is [- 8:7]
  - Find the sum of +4 and +5
  - Find the sum of -4 and -5
- Addition of two numbers of the opposite sign never produces overflow
- Adding two same-signed numbers and obtaining a result of the opposite sign indicates overflow

Overflow

Note that if the number of bits required for the answer is exceeded, overflow will occur. This occurs only if both numbers have the same sign. The overflow will be indicated by an incorrect sign bit.

Two examples are:



Arithmetic Operations with Signed Numbers

Using the signed number notation with negative numbers in 2's complement form simplifies addition and subtraction of signed numbers.

Rules for **addition**: Add the two signed numbers. Discard any final carries. The result is in signed form.

Examples:

00011110 = +30	00001110 = +14	11111111 = –1
00001111 = +15	11101111 = –17	11111000 = –8
00101101 = +45	11111101 = –3	1 11110111 = –9



Subtraction

Rules for **subtraction**: 2's complement the subtrahend and add the numbers. Discard any final carries. The result is in signed form.

Repeat the examples done previously, but subtract:

	00011110	(+30)		00001110	(+14)		11111111	(–1)
–	00001111	–(+15)	–	11101111	–(–17)	–	11111000	–(–8)

2's complement subtrahend and add:

	00011110	= +30		00001110	= +14		11111111	= –1
	11110001	= –15		00010001	= +17		00001000	= +8
	100001111	= +15		00011111	= +31		100000111	= +7

Discard carry

Discard carry

Examples (8 bit numbers)

- **Add 7 and 4 (both positive)**

```
 00000111      7
+00000100     +4
 00001011     11
```

- **Add 15 and -6 (positive > negative)**

```
 00001111      15
+11111010     + -6
1 00001001       9
```

Discard carry →

- **Add 16 and -24 (negative > positive)**

```
 00010000      16
+11101000     +-24
 11111000      -8
```

Sign bit is negative so negative number in 2's complement form

- **Add -5 and -9 (both negative)**

```
 11111011      -5
+11110111     + -9
1 11110010     -14
```

Discard carry →